Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 06

Lecture – 32

Interactive System Development Life Cycle Stages

Hello and welcome to the NPTEL MOOC's course on the Foundation for Virtual and Augmented Reality Systems. We are going to start a new topic today: Interactive System Development Life Cycle. If we reflect on our earlier learning, let us quickly see what we have learned so far. Now, earlier at the very beginning, we talked about AR/VR systems and subsequently we talked about the relationship of these extended reality systems to interactive systems.

So, we talked about the idea of interactive systems first and then learned the relationship between XR systems and interactive systems. Essentially what we have learned is that XR systems are nothing but a class of interactive systems because eventually the users of the XR systems are supposed to be laypersons. Also, we noted how design and development of interactive systems are affected by various human factors and in the context of that knowledge, we also talked about human factors and their relationship to XR systems.

Eventually, that discussion led to the concept of user-centric design. So, when we are talking about building an interactive system, essentially what we try to do is build and design a system that has followed a user-centric design approach. Now, what is user-centric design? That is essentially to involve the users in the design process so that we can understand their needs, their expectations, and their goals. And accordingly, we can design the system so that all those are supported.

If we do not do that, then it will lead to systems that are unacceptable to the users. In this context, we also talked about the usability of a system. And we mentioned that the goal of any interactive system is to make the system usable with respect to a specific group of users in a specified context of use. So that is what we have learned so far. Note that when we talk of usability, we talk of a specific group of users, a specified context of use, and specific tasks.

So it is not that we try to make a system which will be usable for any user in any context for any task. So these things are important. Now the question is, how do we develop such a system? How do we take into account the users in the overall development process?

So, that is what we are going to learn in this and subsequent few lectures. So, the basic thing in the user-centered design approach is to, or in other words, the objective of user-

centered design is to design products that increase usability. Now, in our context, products are nothing but the application software that we are dealing with. Now, we have seen that when we use the term user-centered design or user-centric design or human-centric design or participatory design, so there are different terms, different phrases to represent the same concept.

The basic thing there is that such approaches, in whatever name we call it, such approaches essentially emphasize either active or passive involvement of, the users' involvement of the users in the overall design lifecycle which includes design and implementation. So, the goal is to involve the users either actively or passively. The core design concern when we are talking about user-centric software is designing usable systems to cater to the needs and expectations of layman users or laypersons. So, essentially we are trying to build application software, which are XR systems, which of course depends on some hardware, but that is a separate consideration. Now, this application software is supposed to be interactive software, interactive system, because it is supposed to be used by those who may not know how the hardware works, how the software works. They may not need to know. So, they are the laypersons.

For them, we need to go for user-centered design approach or user-centric design approach or human-centric design approach or participatory design approach or similar such phrases we can use, but all point to a particular approach where we try to include or involve the end users in the design process either actively or passively with the end goal of having usable systems having our application software usable.

Now in order to achieve that goal, in order to achieve the goal of having a usable product, usable software application software, certainly we cannot work in a haphazard manner. We cannot work in an ad hoc manner; we require a systematic approach, a proper step-by-step process we should follow so that at the end the process itself ensures that we get as output of the process the usable application software. Now, this brings us to the idea of software development lifecycle. Probably the majority of you, if not all, are already aware of these things; probably you have studied this in a software engineering course. If you have gone through any such course, then you know what a software development life cycle is. The idea here is that we follow such development life cycles to develop and deploy software so as to comprehensively capture and represent the design and development activities that should go into building the software. So, essentially such life cycles give us a very systematic, structured way of looking at the whole development process.

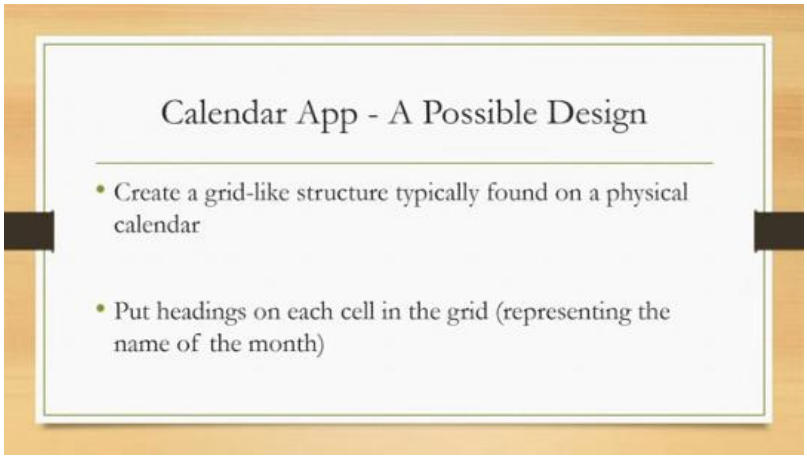
Now, if we can use a single term to represent the idea of software development life cycle, we can use the term software engineering, engineering a software. So, when we say we are trying to engineer a software, we need to follow a software development Or in other words, we need to build the software in some stages. Now, these stages essentially capture or structure the whole process and through their connection, they essentially let us represent in a very structured manner how the development process should take place so that at the end we get what we want to get.

In our context, we want to get a usable software, and we should have a structured software development lifecycle that leads us to a usable software. Now, let us try to understand what is the problem, why we need to follow a development lifecycle. So, when we are trying to

engineer a software towards achieving our goal, we said that we need to follow a a development life cycle as a structured way of achieving our goal. Now, to understand the complexity of the problem and why we need a structured way, let us see a simple, very simple example.

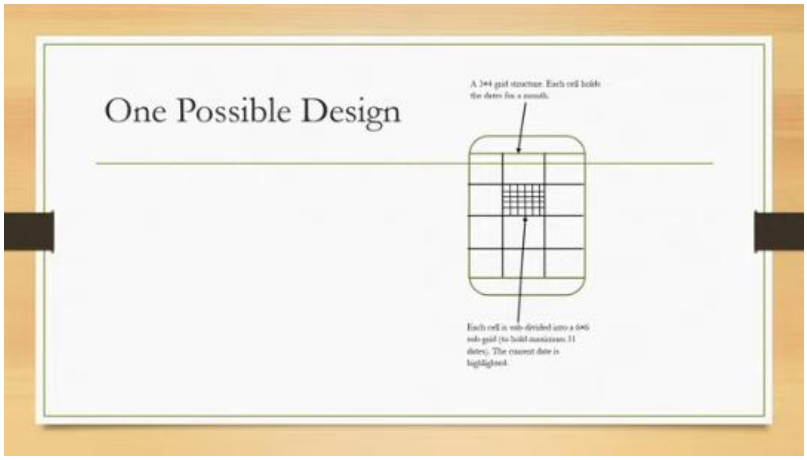
That is, let us try to create a very simple calendar application. We are all familiar with such applications. It comes in our smartphone as a ready-made app. But if we are asked to do this, what should we do? How are we going to approach the problem?

So, we can come up with various ideas. The first thing that comes to our mind when somebody tells us to build a calendar application, a calendar app, is to come up with a design idea, how it should, how the interface should look. So, one such possible design idea can be create a grid-like structure typically found on a physical calendar. So, we have experience dealing with physical calendars.



Because of that experience, that knowledge, we immediately start thinking of something similar in the digital version of the calendar. And this grid-like structure will have headings on each cell of the grid, which represents the name of the month like in a physical calendar. Then we can create sub grids in each cell to hold the dates like again in a physical calendar and render the entire structure on the screen. So that is one design idea.

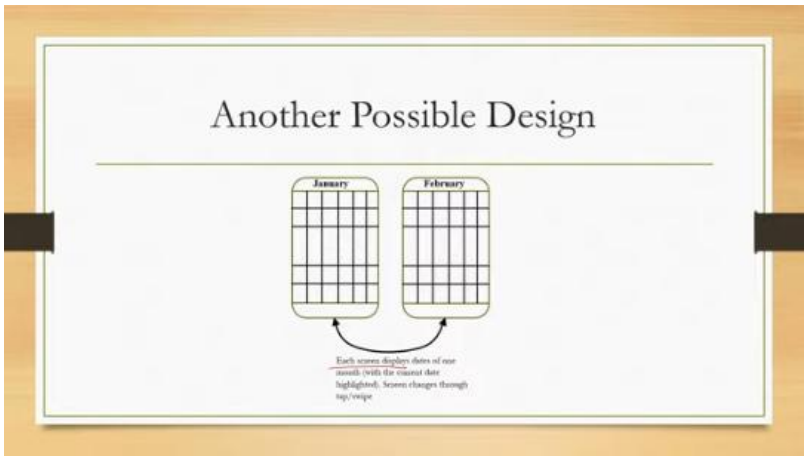
Also, additionally, we may like to add some features which typically are activities associated with calendars such as highlighting a date and time for certain occasions, events as reminders. So, we can highlight the current date and day, current month or we can also highlight some specific dates where we set some reminders for certain activities, some events, some meetings, something like that. So, if we put this into practice, this may look something like this. So, this is our device. We are assuming that we are making the design for a smartphone or mobile device. This is our device. Then we have grid-like structures.



So, there are cells in the grid and each cell is further divided into cells. So, each cell is subdivided into a 6 by 6 subgrid to hold maximum 31 dates as we know in the English calendar and we can highlight the current date. So, we require a 3 by 4 grid structure. Each cell holds the dates for a month. So, each cell represents a month and inside each cell, there are days like shown here in this figure. So, the outer grid structure should be 3 by 4 so that it can hold 12 months. The inner grid structure should be at least 6 by 6 so that 31 days can be shown. Should we stop here? Should we

Assume that whatever we have designed is the only possible design or the best design or the most usable design. Can I draw a conclusion based on this design? These are some questions that come to our mind when we are trying to implement such a design idea. But there can be many more such designs, in fact. This is only one.

So there can be another design where each screen displays the dates of one month. So, at a time only one month is shown on a screen; say, for example, January is shown here and under January there are dates of January shown in this form of a grid. Now, if I want to see the next month or the previous month, we need to swipe next or either swipe left, right, or up, down depending on our design choice.



Here also we can highlight the current date in a month view and screen changes happen through tap or swipe. So, any sort of interaction we can deploy to go to the next month or

the previous month or even the previous year. That can be another design. Now, which one is better? If I ask you this simple question: whether the previous one is better or this one is better, or which one is more usable, the previous one or this one, can we immediately answer that question?

In fact, there can be many such possibilities. It is not that only these two are possible designs. There can be in fact, a very large number of such possible designs. And the challenge is how to choose which one is the right one.

Now, here in our context, the right one means which one is the best in terms of usability. Now, what would be the size of the design space? Design space means the set of all possible designs. So, suppose there are n elements, or n objects to be placed on a screen.

So, we can actually arrange these objects in n factorial ways on a screen. So, that is the typical size of a design space when we have n distinct objects and none of these objects are repeated. If we allow repetition, then certainly the number will increase manifold. And for a suppose calendar type thing where if we go by the first view, then you can try to figure out what is n and what is the total size of the design space. Vis-a-vis, if we go to the next view again, you can find out what is n and what is the size of the design space.

Now the thing is that In general, for a calendar app, we have a fixed N, that is the number of things to be displayed. So, those are month name, then date, possibly year, and the N is very large. So, correspondingly, we can have many, many such designs. In fact, a very large number of designs which are unmanageable or even unthinkable.

Now, the thing is, Clearly, some of these designs, some of all these possible designs in the design space are invalid because dates have to be ordered in a particular way. It is not possible to arrange all dates in random ways. These are some things that come through our experience, knowledge, intuition that if the theoretical design space is n factorial, The majority of such design choices may be invalid because of practical considerations and we are left with few choices.

However, rarely these remaining choices are very small or only one choice is left. In most cases, we can have a reasonably significant number of choices if we are dealing with practical systems. So, although we can immediately invalidate many designs based on our experience and intuition, but still we may be left with 10, 15, 20 choices which seem to be reasonable. And the challenge is to identify the best among all these possible design choices.

Now, to identify the best, we cannot rely on our intuition. For very experienced developers, for very experienced designers, probably that is true, but not for everyone in general, that our intuition may not help us lead to a single best solution. So, we need to follow a systematic approach and that systematic approach is provided by the software development life cycles or SDLC. So, essentially the role of SDLC is to help us identify the best among all possible design, all possible valid design choices following a systematic approach.

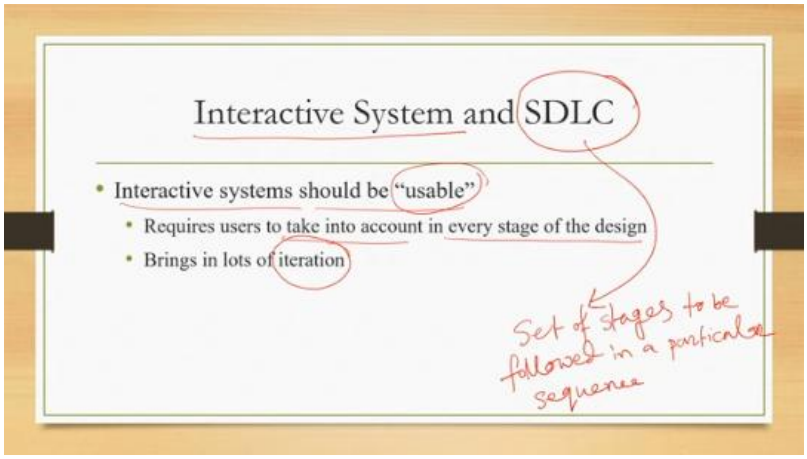
Now, those who have already done a course on software engineering are well aware of different software development life cycles. The classical model is the waterfall model. Of

course, this is not used in practice, but it is primarily used for pedagogical purposes. We have spiral model, evolutionary model, iterative model and so on.



I will not go into the details of any of these models because those are not part of this course. I am sure that the majority of you have already gone through them or if not, you can consult any book on software engineering to get more ideas on software development lifecycle. What we are more interested in here is the interactive systems and the corresponding development lifecycle, software development lifecycle that helps us build usable interactive systems. So, we talked about interactive systems which should be usable. Now, it requires the users to be taken into account in every stage of the design.

So, we need to involve the users in our design process as active or passive participants. Now, that is not an easy thing to include users' needs and users' expectations in our design process. It requires iteration between stages. So, the software development lifecycle is nothing but a set of stages to be followed in a particular sequence. In a particular sequence, ideally these things should be followed.

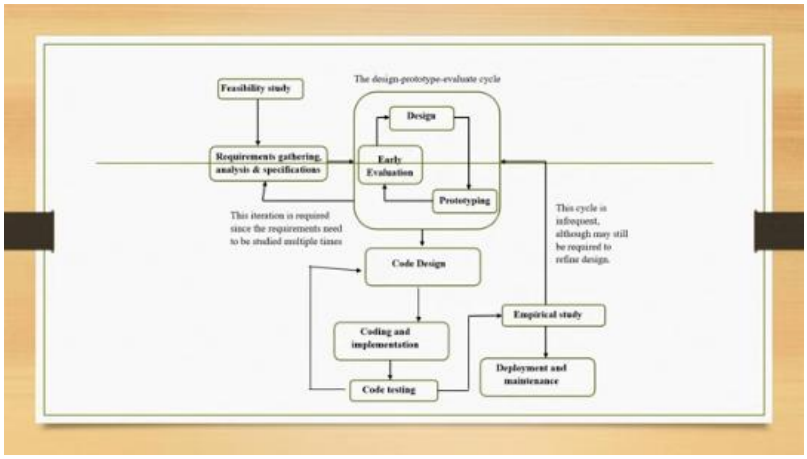


Now, if we are bringing users into the picture, Then these stages, the sequence in which it has to be followed gets altered, gets changed significantly and there appears lots of iteration between stages. So, it is kind of a loop. We complete a stage, go to the next stage, then we

may need to go back to the previous stage rather than proceeding to the next stage in the sequence.

So, that looping happens more frequently when we are dealing with interactive systems and usable systems. So, in order to implement a user-centric design approach for building interactive software in general and extended reality software in particular, we require a highly iterative lifecycle. A highly iterative software development lifecycle. And these iterations between stages, the looping between stages, are there to account for the user feedback. Although I am using the term feedback here, it essentially means involving the users in different ways, not necessarily through feedback only.

So, such an iterative Software development lifecycle for interactive system development may look something like this. As you can see, we have several stages. Feasibility study, then requirement gathering, analysis, and specification. Then there is this design prototype evaluate cycle, which comprises the design stage, prototyping stage, and early evaluation stage.



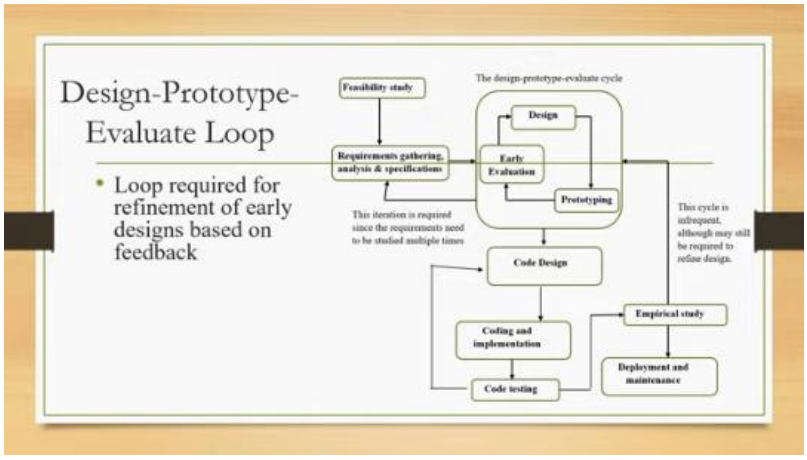
Then we have code design. Coding and implementation, then code testing, empirical study or usability study, and finally deployment and maintenance. And there are iterations, say for example, from this design prototype evaluate cycle to the requirement gathering stage. Then within the design prototype evaluate cycle, there are iterations as shown here between these three stages. Then we have iteration here, we have iteration here.

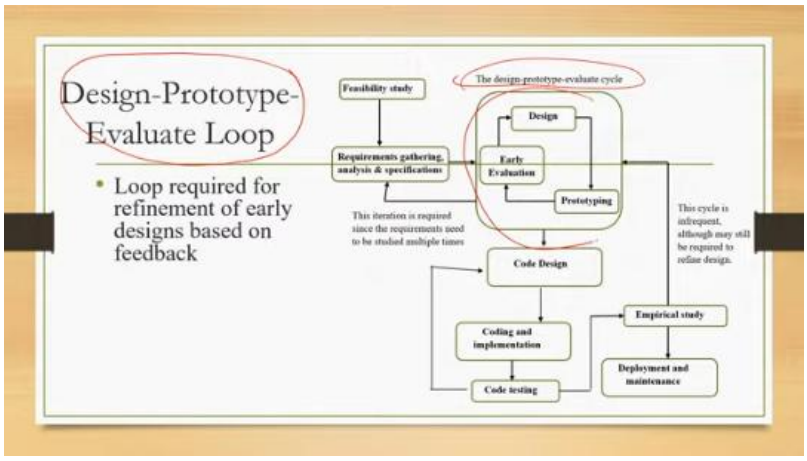
So, it is a pretty good amount of iteration we can see. So, it comprises several stages and several iterations between the stages, and we need to follow such iterative development lifecycle in order to build usable interactive systems. So, let us quickly have a look at what these stages are and what they do. So, let us start with the requirement gathering stage.

We are excluding the feasibility study stage as this is basically whatever is done here is true for any software, not only interactive systems. So this is a common stage where we check for the feasibility of the project in terms of available resources, available money, available time, and so on. More importantly, we should focus on, we will be focusing on the stages where there is a difference between any software and interactive software. So, we start with this requirement gathering stage.

Now, this stage from this very stage itself, we should involve the end users along with the customer or client of our system, which is typically the practice in any software development. Typically when we say requirement analysis, we refer to the requirements of the client. Now in the case of interactive systems, along with client requirements, we need to take into account the requirements of the end users. So from this stage onwards, we need to involve the users. There are several methods to gather end user requirements such as contextual inquiry, ethnographic studies, cultural probes, and so on.

So, these are part of this stage. Then we have the design prototype evaluate loop as shown here. This entire loop is the design prototype evaluate loop. This loop is required for the refinement of early designs based on user feedback. So, we have involved the users.



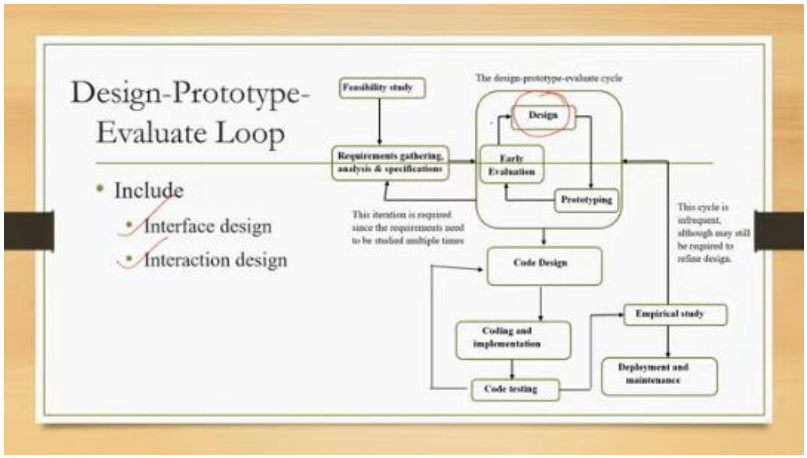


So, whatever feedback they give, based on that, we can modify the design and we need to do it in several iterations. So, suppose based on this requirement analysis, we came up with a design idea like our calendar app. Then we can quickly create a prototype of this idea and get it quickly evaluated. Then we identify problems through evaluation. Now, based on those problems we refine the design, then again evaluate, then again prototype, evaluate and this process continues till we reach a

stage where not many significantly different problems are being identified through this evaluation. So, in that case, we can say that our design has stabilized and we can move to

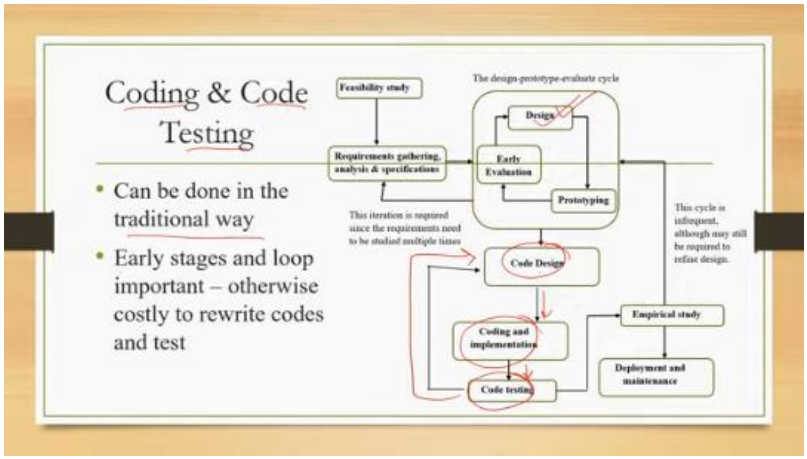
the next stage. So this cycle involves many iterations. So it is essential that each of these stages can be completed quickly so that many iterations happen within a reasonable time. So it should be performed quickly.

The whole cycle should be less time-consuming. Otherwise, the overall turnaround time will increase significantly. Now, this design-prototype-evaluate loop includes interface design and interaction design, these two activities. So, when we say that we are trying to design the system, essentially there are two parts. We are designing interface and interaction when we talk of design here in this design-prototype-evaluate cycle.

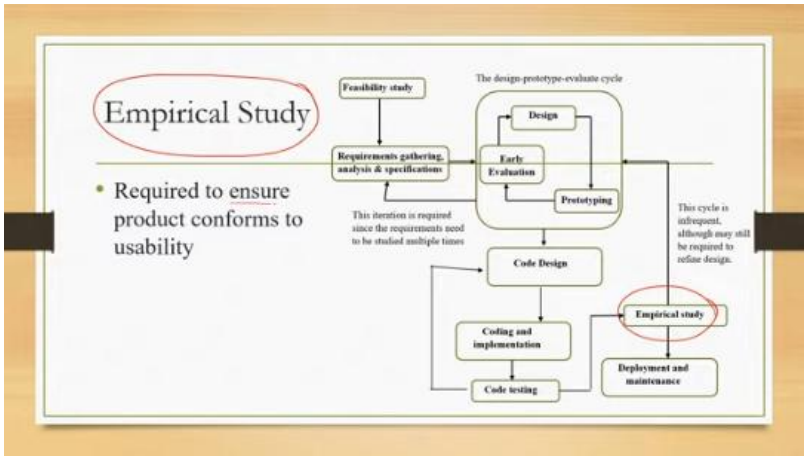


Later on, there is another design part that is code design, which is separate from the design that we have mentioned in the design-prototype-evaluate cycle. So the design-prototype-evaluate cycle primarily deals with design of the interface and interaction from the user's point of view based on user requirements. Then when it comes to coding and code testing, It can be done in the traditional way as is typically done in any software engineering process. Of course, here we have code design, then coding and implementation, then code testing.

And through testing, if we find some issues, then we can go back to our code design part. So there is one iteration here. However, the fewer these number of iterations, the better because code design is a very highly skilled process that requires skilled manpower and is costly. So, it is better to identify problems. It is better to properly design at this stage so that later on problems are fewer.



Otherwise, it is very costly to rewrite codes and test. So, that we have to keep in mind. Then comes the empirical study part. This is another important stage which differentiates any interactive system from that of any other software system. So, this stage is required to ensure that the product conforms to usability.



Note that here we have one. So, at three places in this iterative software development life cycle, we have evaluation. One is early evaluation that is As which is part of this design prototype evaluate cycle, then we have code testing or evaluation of the code which is part of coding and implementation, and thirdly, empirical study. Now, this early evaluation is kind of evaluating the ideas of interface and interaction design quickly with fewer resources and less time.

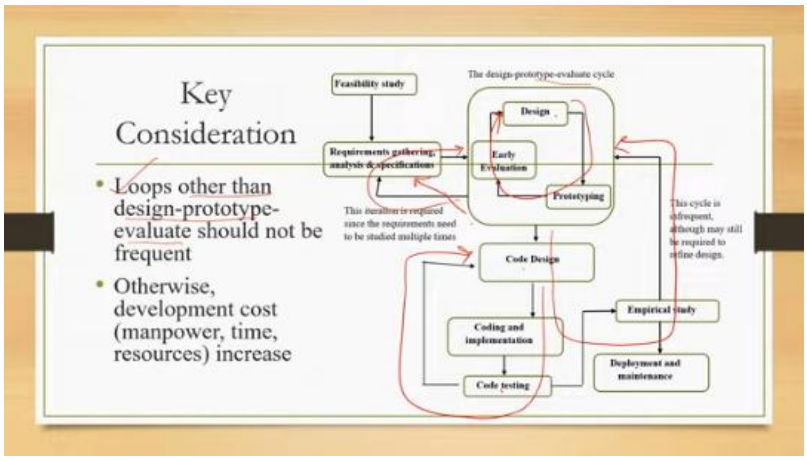
Code testing tests the executability of the code. That means through this testing, we ensure that there are no bugs, and the code executes properly. However, this testing, code testing, does not tell us whether the system is going to be usable. It tells us that the system is executable, the code is executable, bug-free, but it does not tell us whether it is usable from the end user's point of view. That happens when we perform empirical studies.

So, the empirical study stage is required to ensure the product conforms to usability. Now, this involves systematic testing with the end users, which is totally different from how we test our code. Code testing is a different activity, and empirical study or usability testing

with end users is a different activity. And this empirical study is unique. to the interactive system development.

It is typically not part of any other software development where end users are not involved. So, the key considerations in this overall iterative design is that the loops that are there, other than the design prototype evaluate cycle, should not be frequent. So, we have loops at several places. For example, between this design prototype evaluate cycle and the requirement gathering. So, we have this loop.

Then we have a loop between code design and code testing as shown here. We also have a loop between design prototype evaluate cycle and code coding and implementation and empirical studies so this loop and also we have loop within this design prototype evaluate cycle as shown here at several places we have loop now this only this loop design prototype evaluate early evaluate only this loop is designed in a way such that they require less time, and can be performed quickly.



Other loops, particularly the loop between code design, coding and code testing or the loop between design prototype evaluate cycle, through coding and empirical study, these loops if they happen frequently then that will have a significant bearing on the overall outcome of the project, the turnaround time of the project. It may lead to abandonment of the project as well. So, we should be very careful while completing the stages in such cases so that the requirement to go back to previous stages does not even arise.

Otherwise, it will lead to escalation in development cost in terms of manpower, time, resources, and even may lead to abandonment of the project. So that is the overall idea of what we should do to build interactive systems which are usable. And in our case, we are talking of extended reality systems which are usable. So we have learned the idea of ISDLC, that is interactive system development lifecycle.

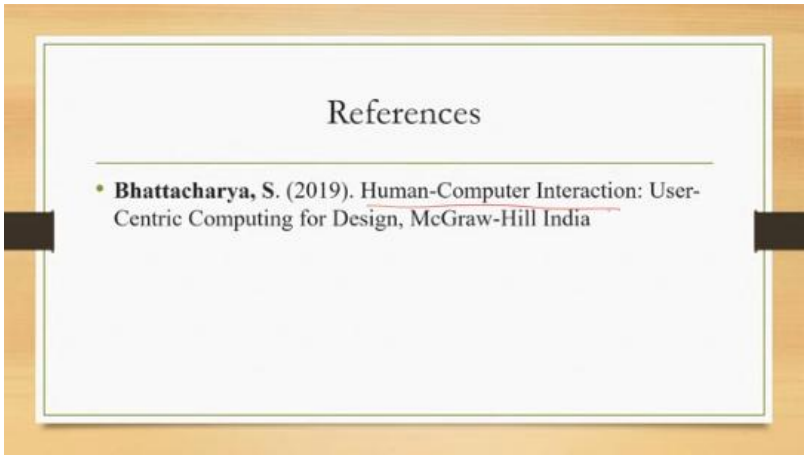
We have seen the stages of the lifecycle broadly comprising of requirement analysis stage, design prototype evaluate loop, then coding and implementation stage and empirical study or empirical testing stage. Earlier we talked about interactive systems, what they are, what usability is, and what the need is for having a usable system. So, when you learn this lecture side by side or go through this discussion side by side, you will get a clearer picture of

interactive systems and how to develop those with respect to our basic concern that is extended reality systems.

In subsequent few lectures, what we are going to do is to focus on the requirement gathering interface interaction design and evaluation aspects of interactive system development because as we have mentioned the feasibility study or the code testing phase are anyway standard phases. So, there is not much difference between how good regular software is built following these stages and how interactive software is built. The difference is in the requirement gathering stage, design prototype evaluate stage, and empirical study stage.

So, in the next few lectures, we are going to devote some time to a discussion of these stages, and we are going to use a case study to understand these stages with respect to our extended reality systems. In the immediately next lecture, we are going to introduce the case study, whatever example we are going to talk about. We are going to introduce that idea and then in subsequent lectures, we will explain each of these stages in terms of that case study and how we have taken care of these concerns through our design effort with respect to that case study we are going to discuss so that you get a hands-on

of a practical system and the theoretical concepts. So, to understand these topics better, you can refer to this book, Human-Computer Interaction, User-Centric Computing for Design. Here the interactive system development life cycles, the stages are discussed in more detail with respect to broader interactive system development concerns. So, if you are interested to know more, please refer to this book. So, with that, we have come to the end of this lecture.



I hope you could follow the concepts and I am looking forward to meeting you all in the next lecture. Until then, thank you, goodbye, and happy learning.